SKITII PRODUCT STATION

Raspberry Pi Station Setup

Repeatable runbook to image / configure another Raspberry Pi 5 kiosk the same way as the lab station.

APP REPO

`Skitii-Desktop-Ble` (Tauri 2 + React + native BlueZ BLE)

PLATFORM

Raspberry Pi 5 · Bookworm · labwc

BRAND COLOR

`#0e1a2d`

LAST UPDATED

2026-08-08

SCOPE

Kiosk · Splash · Plymouth · BLE build

1. What this station is

Layer	Role
Raspberry Pi OS (Bookworm, Wayland / **labwc**)	OS + compositor
`swaybg`	Skitii splash / solid brand fill behind the app
Plymouth theme `skitii`	Boot splash (logo + brand bg)
`~/kiosk.sh`	Forever-loop: hide desktop chrome, keep splash, run app
`~/apps/skitii-desktop-ble/skitii-desktop-ble`	Fullscreen Tauri station binary
UAT GraphQL	Default `.env` → `https://api.hrvardevice.uat.skitii.com/graphql`

Not used for branding: Pi 5 EEPROM white/red “Configure this Raspberry Pi” screen (firmware). Keep `NET_INSTALL_ENABLED=0`. A short black gap before Plymouth is normal.

---

2. Prerequisites (new Pi)

Hardware

- Raspberry Pi 5 (lab unit was 4GB)
- Display (station panel; app targets ~1280×800 / fullscreen)
- Network (Wi-Fi or Ethernet) + SSH access from a Linux build PC

OS image

- Raspberry Pi OS **Bookworm** with desktop (labwc)
- Enable SSH, set user `pi`, configure Wi-Fi before first field deploy
- Prefer SSH key auth from the build PC (`BatchMode=yes`)

Disk space

- Native Tauri release build needs **~3–4 GB free** during compile (`src-tauri/target`)
- Installed binary alone is ~**19 MB**
- Clear `~/Skitii-Desktop-Ble/src-tauri/target` after install if disk is tight (keep the installed binary)

Packages (on Pi)

```
sudo apt-get update
sudo apt-get install -y \
  build-essential curl git \
  libwebkit2gtk-4.1-dev libgtk-3-dev librsvg2-dev libssl-dev \
```

```
  libayatana-appindicator3-dev \
  bluez dbus \
  swaybg \
  plymouth plymouth-themes \
  rsync
```

Node (on Pi)

```
# Node 20+ recommended; use nodesource or nvm if distro node is too old
node -v
npm -v
```

Rust (on Pi)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustc -V
cargo -V
# Always use: export PATH="$HOME/.cargo/bin:$PATH"
```

Build PC

- Linux x86_64 with `rsync` / `ssh` / `scp`
- Source checkout of `Skitii-Desktop-Ble`
- Cross-compile of Tauri+WebKit to aarch64 is **not** set up — **build on the Pi**

---

3. Source tree & scripts (canonical)

All automation lives in the app repo:

```
Skitii-Desktop-Ble/
  .env                          # VITE_* for UAT GraphQL, timings
  scripts/
    kiosk.sh                    # App loop (install → ~/kiosk.sh)
    start-kiosk.sh              # Safe manual start (splash first)
    stop-kiosk.sh               # Stop app only; keep swaybg
    labwc-autostart             # User ~/.config/labwc/autostart (NO kiosk here)
    etc-xdg-labwc-autostart     # System /etc/xdg/labwc/autostart (owns kiosk)
    pcmanfm-lxde-pi/desktop-items-0.conf
    splash/                     # Generated Skitii splash PNGs
    plymouth/                   # skitii.plymouth + skitii.script
    deploy-kiosk-no-flash.sh    # Wallpaper / labwc / no-flash deploy
    deploy-skitii-splash.sh     # Splash + Plymouth deploy
    deploy-pi-osk-scroll.sh     # Rsync + rebuild example
```

Important: RPi OS runs `labwc -m` (merge). **Both** `/etc/xdg/labwc/autostart` and `~/.config/labwc/autostart` run. Only the **system** file should start `kiosk.sh`. User file: kanshi + swaybg only.

---

4. New Pi — ordered checklist

A. Base OS

1. Flash Bookworm, boot, finish first-run wizard **once** (then disable later if needed).
2. `sudo apt update && sudo apt full-upgrade -y`
3. Install packages + Node + Rust (section 2).
4. Confirm Wayland: `echo $XDG_SESSION_TYPE` → `wayland`; socket `/run/user/1000/wayland-0`.

B. Firmware / boot (branding-related)

```
# Hide firmware rainbow (config.txt)
# Ensure in /boot/firmware/config.txt:
#   disable_splash=1

# Hide EEPROM network-install UI (white/red "Configure this Raspberry Pi")
sudo rpi-eeprom-config > ~/bootloader-config.bak
# Edit copy: set NET_INSTALL_ENABLED=0 (keep other keys)
sudo rpi-eeprom-config --apply ~/bootloader-config.edited
sudo reboot   # required for EEPROM
```

Verify after reboot:

```
sudo rpi-eeprom-config | grep NET_INSTALL
# NET_INSTALL_ENABLED=0
```

Tradeoff: `0` ⇒ short black gap before Plymouth. `1` ⇒ Pi configure screen fills gap (not Skitii-branded).

C. Sync app source

From the build PC (`PI_HOST` = new Pi):

```
export PI_HOST=pi@<NEW_PI_IP>
rsync -az -e 'ssh -o BatchMode=yes' \
  --exclude node_modules --exclude src-tauri/target --exclude dist \
  --exclude dist-bundles --exclude .git --exclude tmp-vid-frames \
  /path/to/Skitii-Desktop-Ble/ \
  "$PI_HOST:~/Skitii-Desktop-Ble/"
```

D. Build & install binary (on Pi)

```
export PATH="$HOME/.cargo/bin:$PATH"
cd ~/Skitii-Desktop-Ble
# Ensure .env is correct for this station (UAT vs prod GraphQL)
npm ci   # or npm install
npm run build
cd src-tauri
cargo build --release --bins --features tauri/custom-protocol
cd ..

# CRITICAL: never install a truncated binary (seen as exactly 8 MiB → SIGSEGV)
ls -lh src-tauri/target/release/skitii-desktop-ble
# Expect ~19M (must be >= ~15M)
file src-tauri/target/release/skitii-desktop-ble
# Must NOT say "missing section headers"

mkdir -p ~/apps/skitii-desktop-ble
cp -f src-tauri/target/release/skitii-desktop-ble /tmp/skitii-desktop-ble.new
sync
install -m 755 /tmp/skitii-desktop-ble.new ~/apps/skitii-desktop-ble/skitii-desktop-ble
sync
ls -lh ~/apps/skitii-desktop-ble/skitii-desktop-ble
```

Or use repo helper (after rsync):

```
# On build PC — rebuilds on Pi and restarts kiosk (older helper):
bash scripts/deploy-pi-osk-scroll.sh
# Prefer start-kiosk.sh after install for no-flash restarts.
```

E. Kiosk + no-flash wallpaper

From build PC:

```
export PI_HOST=pi@<NEW_PI_IP>
# Needs passwordless sudo OR edit script to use sudo -S carefully
bash scripts/deploy-kiosk-no-flash.sh
```

What it installs:

- `~/kiosk.sh`, `~/start-kiosk.sh`, `~/stop-kiosk.sh`
- `~/.config/labwc/autostart` (splash only; **no** kiosk)
- `/etc/xdg/labwc/autostart` (swaybg + **kiosk**)
- pcmanfm desktop solid `#0e1a2d`
- Overwrites `/usr/share/rpd-wallpaper/*` with dark/splash asset (backup `*.bak-skitii`)

- `disable_splash=1` in `config.txt` if missing

F. Skitii splash + Plymouth

```
export PI_HOST=pi@<NEW_PI_IP>
export PI_SUDO_PASS='...'   # only if script uses sudo -S
bash scripts/deploy-skitii-splash.sh
```

What it installs:

- `~/skitii-splash.png` (1920×1080 brand splash)
- Plymouth theme `/usr/share/plymouth/themes/skitii/`
- `Theme=skitii` in `/etc/plymouth/plymouthd.conf`
- `splash` (+ `quiet`) on `/boot/firmware/cmdline.txt`
- Greeter wallpaper → splash
- Restarts kiosk with `swaybg -i ~/skitii-splash.png -m fill`

sudo -S warning: Never pipe password into `tee` for writing `cmdline.txt` / conf files (can overwrite the file with the password). Prefer `sudo bash -c 'printf … > file'` or the helpers in `deploy-skitii-splash.sh`.

Reboot once so Plymouth initramfs picks up the theme:

```
sudo reboot
```

G. Station identity

- Physical device id: localStorage key `skitii_physical_device_id` (`SKT-…`)
- Register the device in admin if backend binding is required

---

5. Day-to-day commands

```
# SSH
ssh pi@<PI_IP>

# Manual stop / start (ALWAYS use these — not raw pkill)
bash ~/stop-kiosk.sh
bash ~/start-kiosk.sh

# Status
pgrep -af 'swaybg|kiosk|skitii-desktop|pcmanfm|piwiz'

# Logs
tail -f ~/skitii-ble-run.log
```

```
# Escape fullscreen (USB keyboard on device)
# Ctrl+Shift+Esc  or  Ctrl+Shift+Q  (as implemented in app)
```

Rebuild after code change

```
# Build PC
rsync …  # as in section 4C
# Pi
export PATH="$HOME/.cargo/bin:$PATH"
cd ~/Skitii-Desktop-Ble
npm run build
cd src-tauri && cargo build --release --bins --features tauri/custom-protocol
# verify size >= 15M, then install + sync as in 4D
bash ~/start-kiosk.sh
```

EEPROM toggle (optional)

```
# Show
sudo rpi-eeprom-config | grep NET_INSTALL

# Set NET_INSTALL_ENABLED=0 or 1 via edit +:
sudo rpi-eeprom-config --apply /path/to/edited.conf
sudo reboot
```

6. Key paths on the Pi

Path	Purpose
`~/apps/skitii-desktop-ble/skitii-desktop-ble`	Installed app
`~/Skitii-Desktop-Ble/`	Source + build tree
`~/kiosk.sh`	Autostart loop
`~/start-kiosk.sh` / `~/stop-kiosk.sh`	Manual control
`~/skitii-splash.png`	Desktop / swaybg splash
`/etc/xdg/labwc/autostart`	System autostart (**starts kiosk**)
`~/.config/labwc/autostart`	User autostart (splash only)
`/usr/share/plymouth/themes/skitii/`	Boot theme
`/etc/plymouth/plymouthd.conf`	`Theme=skitii`
`/boot/firmware/config.txt`	`disable_splash=1`
`/boot/firmware/cmdline.txt`	must include `root=…` and `splash`
`/etc/lightdm/pi-greeter.conf`	Greeter bg / wallpaper

7. Boot sequence (expected)

1. Power on → brief black (firmware; network-install off)
2. Plymouth **Skitii** splash (waveform + white **SKITII** on `#0e1a2d`)
3. labwc → `swaybg -i skitii-splash.png` → `kiosk.sh` → fullscreen app
4. Manual stop → splash remains; start → splash first, then app

8. Troubleshooting

Symptom	Likely cause	Fix
App not showing; dark/splash only	Truncated binary (~8 MiB) / SIGSEGV	Rebuild; verify size ≥15M; `install` + `sync`; restart kiosk
White/red "Configure this Raspberry Pi"	`NET_INSTALL_ENABLED=1` or stuck in bootloader	Set `NET_INSTALL_ENABLED=0` + reboot; if stuck forever, check SD / `BOOT_ORDER`
RPi desktop wallpaper flashes on restart	pcmanfm / wrong start order	Use `stop-kiosk.sh` / `start-kiosk.sh`; ensure splash painted **before** killing app; system autostart has no pcmanfm
Two kiosk loops	User + system autostart both start `kiosk.sh`	Comment kiosk out of `~/.config/labwc/autostart`
White flash when app opens	WebKit default bg	App has `backgroundColor: #0e1a2d` in `tauri.conf.json` — rebuild required for that fix
Plymouth not showing	No `splash` in cmdline / theme not in initramfs	Add `splash`; `plymouth-set-default-theme -R skitii` or `update-initramfs -u`
`cmdline.txt` is only a password	`sudo -S | tee` bug	Restore from `/proc/cmdline` / backup; never tee with sudo -S stdin
BLE fails	BlueZ / permissions	`bluetoothctl`; user in `bluetooth` group; adapter up
Disk full on build	`target/` growth	`rm -rf ~/Skitii-Desktop-Ble/src-tauri/target`; `sudo apt-get clean`

Binary health check

```
ls -lh ~/apps/skitii-desktop-ble/skitii-desktop-ble
file ~/apps/skitii-desktop-ble/skitii-desktop-ble
# Bad: "missing section headers" or size exactly 8388608 (8 MiB)
timeout 3 ~/apps/skitii-desktop-ble/skitii-desktop-ble
# Bad: Segmentation fault (139)
```

9. Env / product notes

From `Skitii-Desktop-Ble/.env` (example UAT):

```
VITE_GRAPHQL_URL=https://api.hrvardevice.uat.skitii.com/graphql
VITE_BASELINE_SECONDS=30
VITE_MEASUREMENT_PHASE_SECONDS=0
```

```
VITE_SESSION_MAX_SECONDS=1200
VITE_HR_GRACE_SECONDS=10
```

Change GraphQL URL for production builds before `npm run build`.

App README: `Skitii-Desktop-Ble/README.md`.

---

10. Related deploy scripts (quick reference)

Script	When to run
`scripts/deploy-pi-osk-scroll.sh`	Rsync + on-Pi rebuild + install
`scripts/deploy-kiosk-no-flash.sh`	Kiosk scripts + wallpaper / labwc hardening
`scripts/deploy-skitii-splash.sh`	Splash PNGs + Plymouth + greeter

Set `PI_HOST=pi@x.x.x.x` for any new station.

---

11. Security notes (field units)

- Change default OS password; use SSH keys; disable password SSH if possible
- Do not commit real station passwords into git
- Prefer VPN / private LAN; do not expose SSH publicly
- EEPROM / boot backups on Pi: `~/bootloader-config.bak*`

---

12. One-page “clone station” summary

```
1. Flash Bookworm → SSH → apt packages + Node + Rust
2. NET_INSTALL_ENABLED=0 + disable_splash=1 → reboot
3. rsync Skitii-Desktop-Ble → Pi
4. npm build + cargo release → verify ~19M → install to ~/apps/...
5. deploy-kiosk-no-flash.sh
6. deploy-skitii-splash.sh → reboot
7. Verify: swaybg -i splash, kiosk.sh, skitii-desktop-ble
8. Register physical device id in admin
```

Ops forever after:

```
bash ~/stop-kiosk.sh
bash ~/start-kiosk.sh
```